



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT,Vellore.



Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively
3. To synthesize efficient algorithms in various engineering design situations



BCSE204L- Design and Analysis of Algorithms-

Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	Design Paradigms: Greedy, Divide and Conquer Techniques	6 hours
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	String Matching Algorithms	5 hours
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	Graph Algorithms	6 hours
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - <u>Network Flows</u> : Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, <u>Push Re-label Algorithm</u> – Application of Max Flow to maximum matching problem		
Module:5	Geometric Algorithms	4 hours
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	Randomized algorithms	5 hours
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	Classes of Complexity and Approximation Algorithms	7 hours
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	Contemporary Issues	2 hours



BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, Éva Tardos ,Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K.Ahuja, Thomas L.Magnanti and James B.Orlin, “ Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.



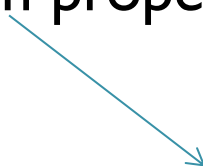
Module 4: Graph Algorithms(6 Hours)

- All Pair Shortest Algorithm
 - Bellman Ford Algorithm
 - Floyd - Warshall Algorithm
- Network Flows – Flow Networks
- Maximum Flow
 - Ford Fulkerson
 - Edmond Karp
 - **Push Re-label Algorithm**



Push Re-label Algorithm

- Another approach to compute Maximum Flow.
- Better than Ford-Fulkerson Method- runs in $O(V^2E)$ time.
- Fastest Maximum Flow Algorithm.
- Rather than examine the entire residual network to find an augmenting path, **push-relabel algorithms** work on one vertex at a time, looking only at the vertex's neighbour's in the residual network.
- Push-relabel algorithms do not maintain the flow-conservation property throughout their execution.



difference between total outflow and total inflow for every vertex (Except source and sink) is maintained 0



Push Re-label Algorithm - Properties

- **Preflow:** $f: V \times V \rightarrow \mathbb{R}$ that satisfies the capacity constraint and the following relaxation of flow conservation.

$$\sum_{v \in V} f(v, u) - \sum_{v \in V} f(u, v) \geq 0$$

for all vertices $u \in V - \{s\}$. That is, the flow into a vertex may exceed the flow out. We call the quantity

$$e(u) = \sum_{v \in V} f(v, u) - \sum_{v \in V} f(u, v)$$

the *excess flow* into vertex u . The excess at a vertex is the amount by which the flow in exceeds the flow out. We say that a vertex $u \in V - \{s, t\}$ is *overflowing* if $e(u) > 0$.

Allows inflow to exceed the outflow before reaching the final flow. In final flow, the net difference is 0 for all except source and sink.



Push Re-label Algorithm

- In fluid flow problem- Here consider edges as water pipes and nodes are joints.
- Source is Considered to be at the **highest level** and sends water to all adjacent nodes.
- Once a node has excess water, it pushes water to a smaller height node.
- If water gets locally trapped at a vertex, **the vertex is relabelled which means its height is increased.**
- **3 divisions**
 - **Pre-processing**- Source- Highest Point, Sink and all other nodes are at the lowest point.
 - **Push Operation** – sending flow to out-neighbours
 - **Relabel Operation**- Increasing height of a node is called relabel operation.

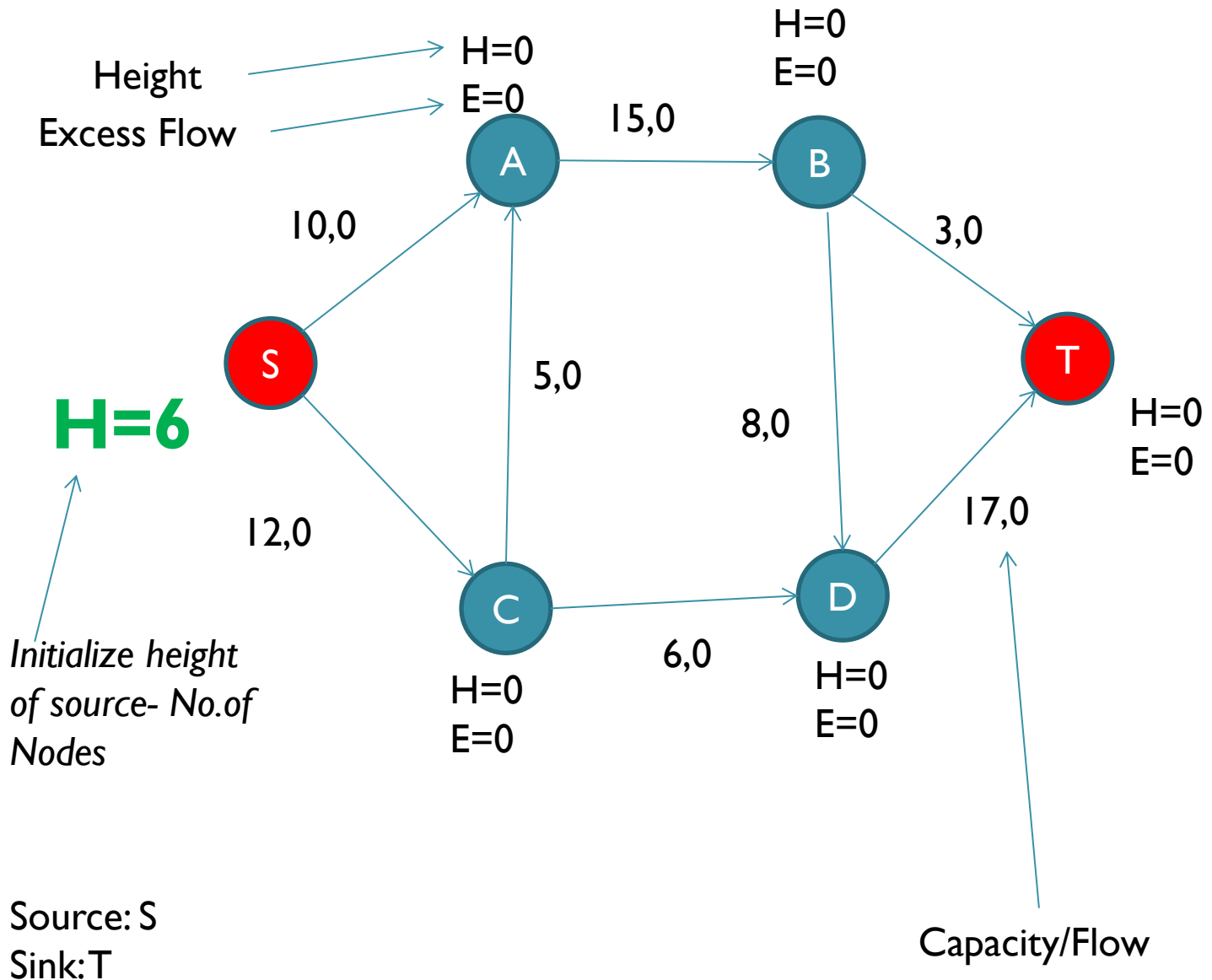


Push Re-label Algorithm

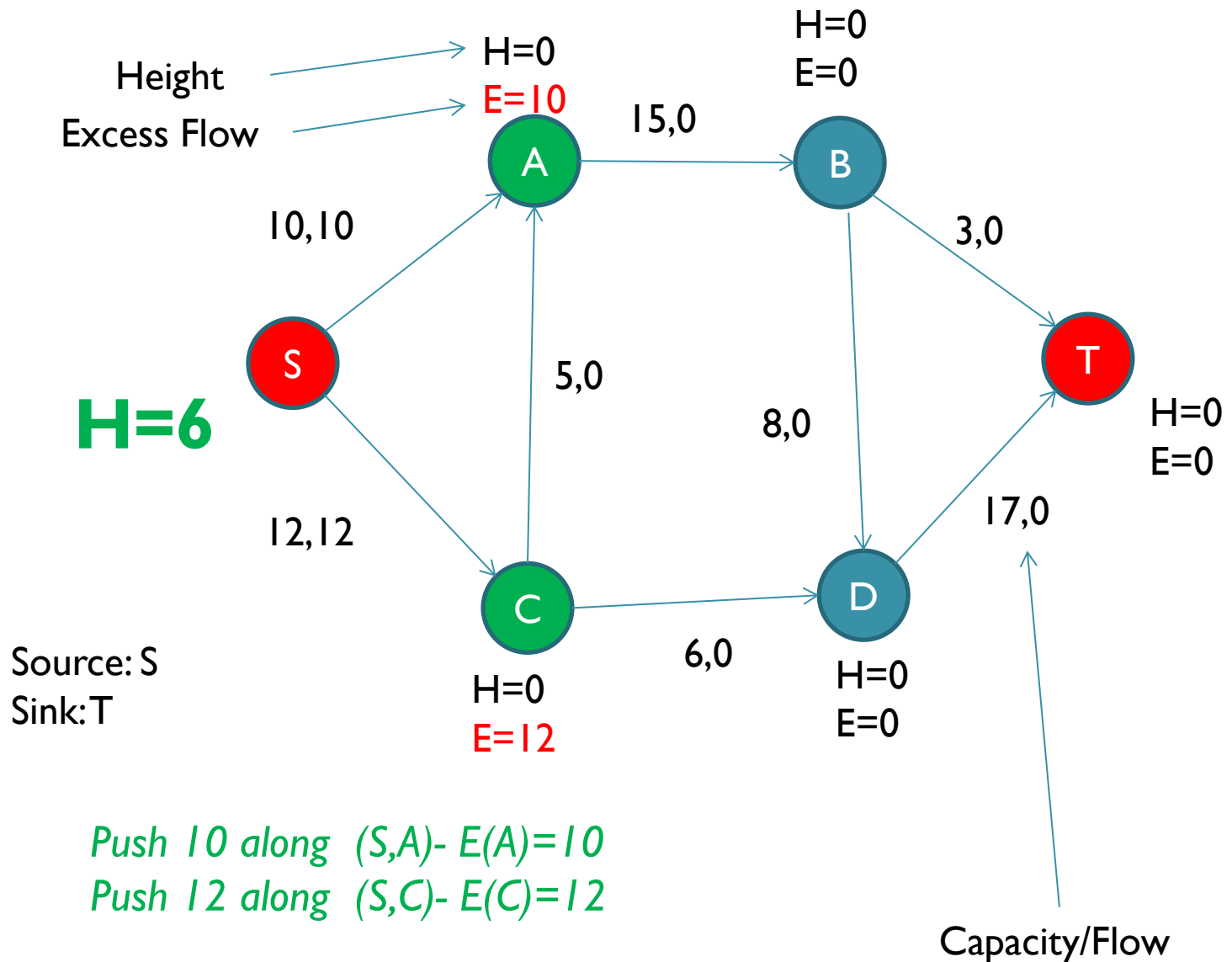
- Each vertex has associated to it a **height variable** and a **Excess Flow**.
- **Height** is used to determine whether a vertex can push flow to an adjacent or not.
- **Excess flow** is the difference of total flow coming into the vertex minus the total flow going out of the vertex.
- A node is allowed (Temporarily) to have more flow coming into it than flow going out.
 - Means a node can have “excess flow”
 - But the edges must respect the capacity condition.
 - Source can have arbitrary amount of excess flow.
- A node is said to be “active” if it has excess flow in it.



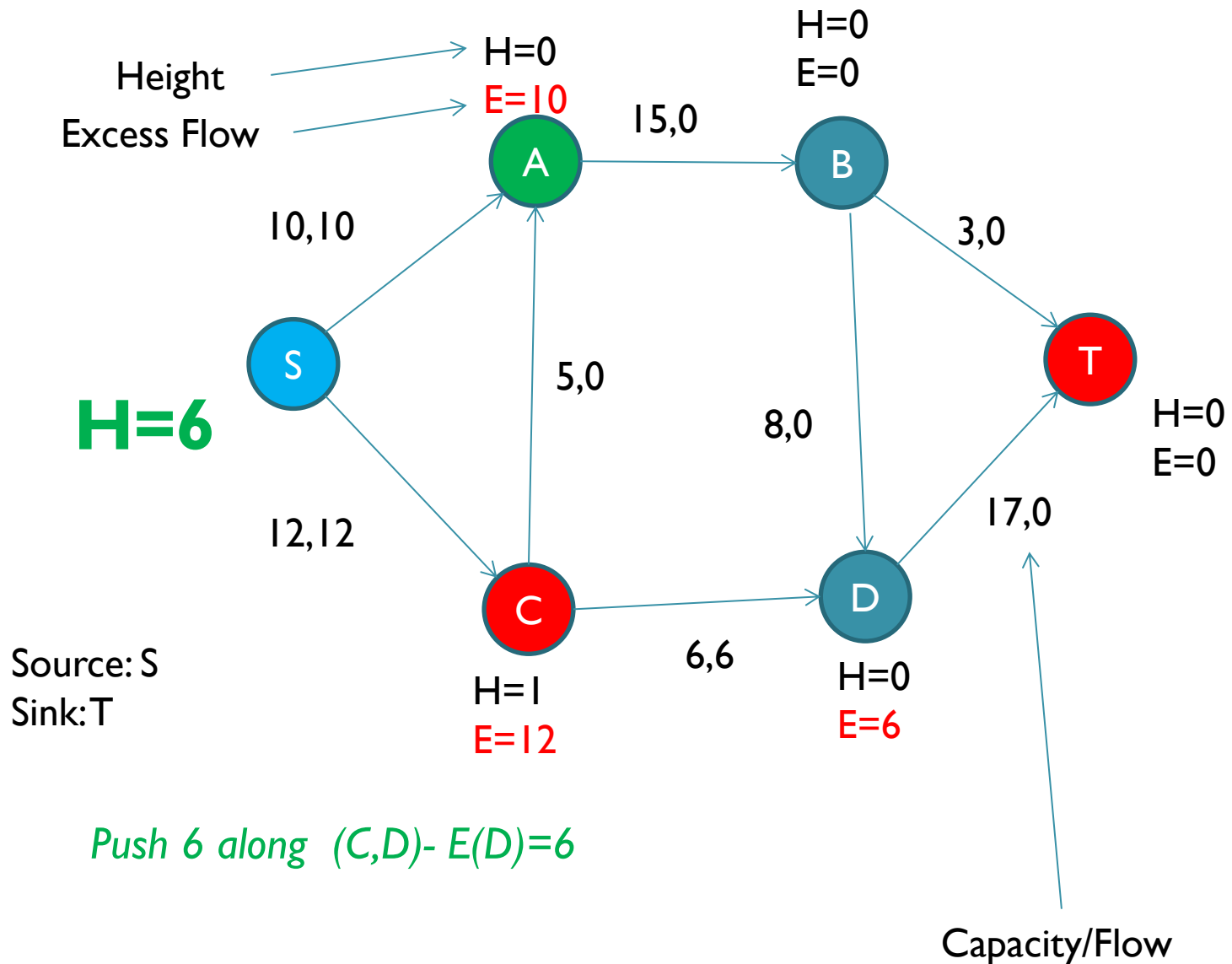
Maximum Flow –Push Relabel- Step-I Pre-processing



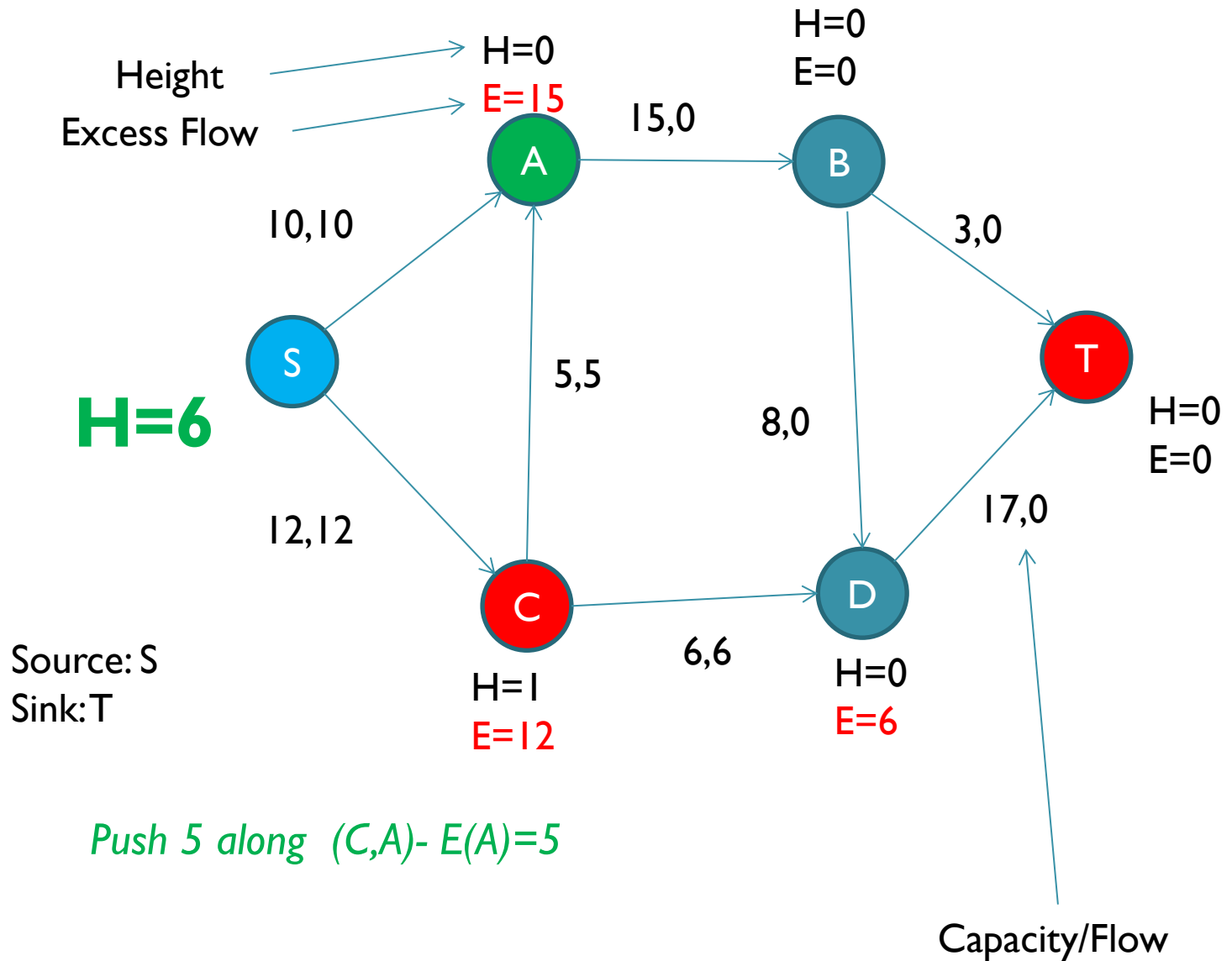
Maximum Flow – Push Relabel- Push Operation – Step II



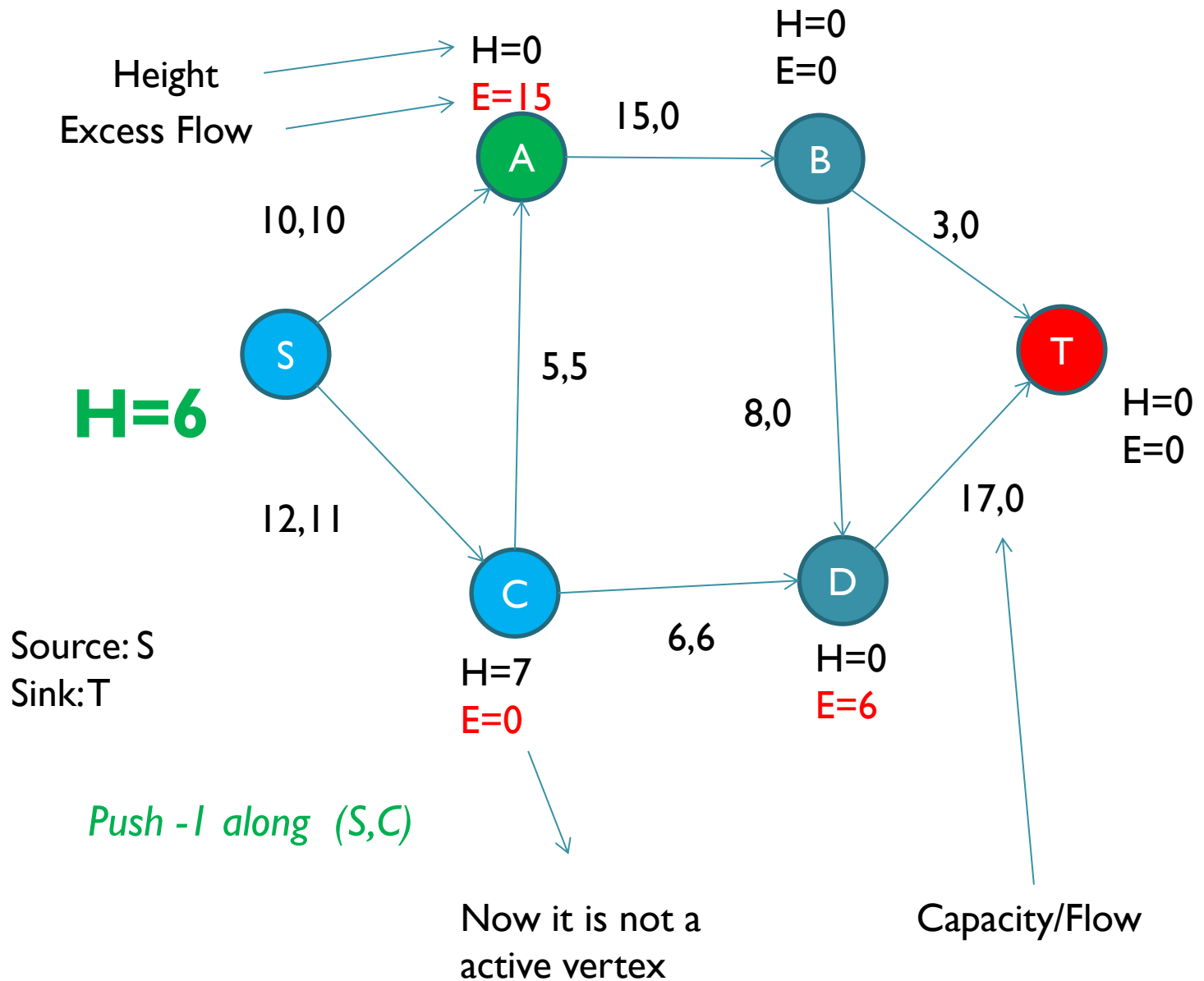
Push Relabel- Step-3- RELABEL OPERATION on "C" WITH $H=1$



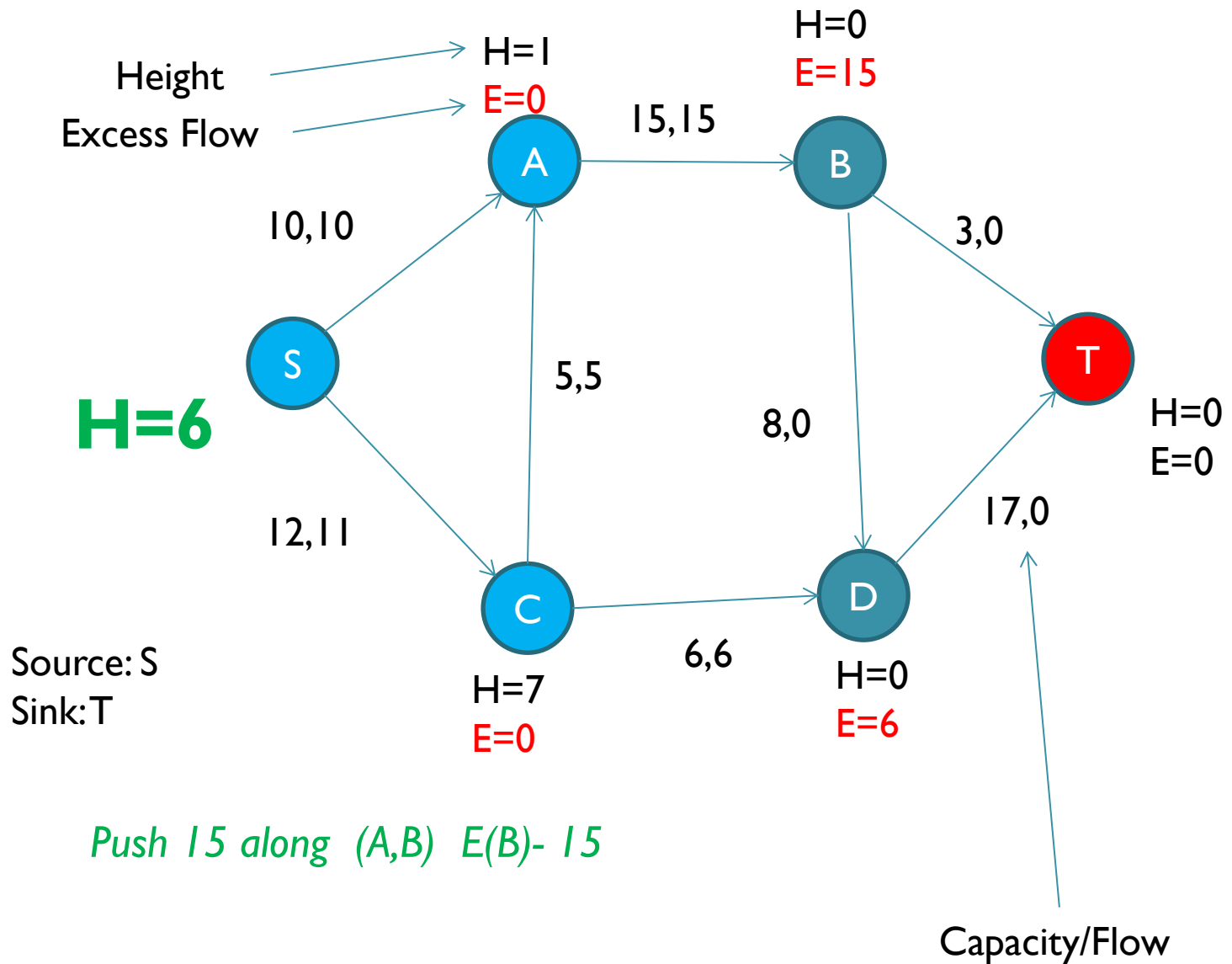
Push Relabel- PUSH OPERATION on C WITH $H=1$



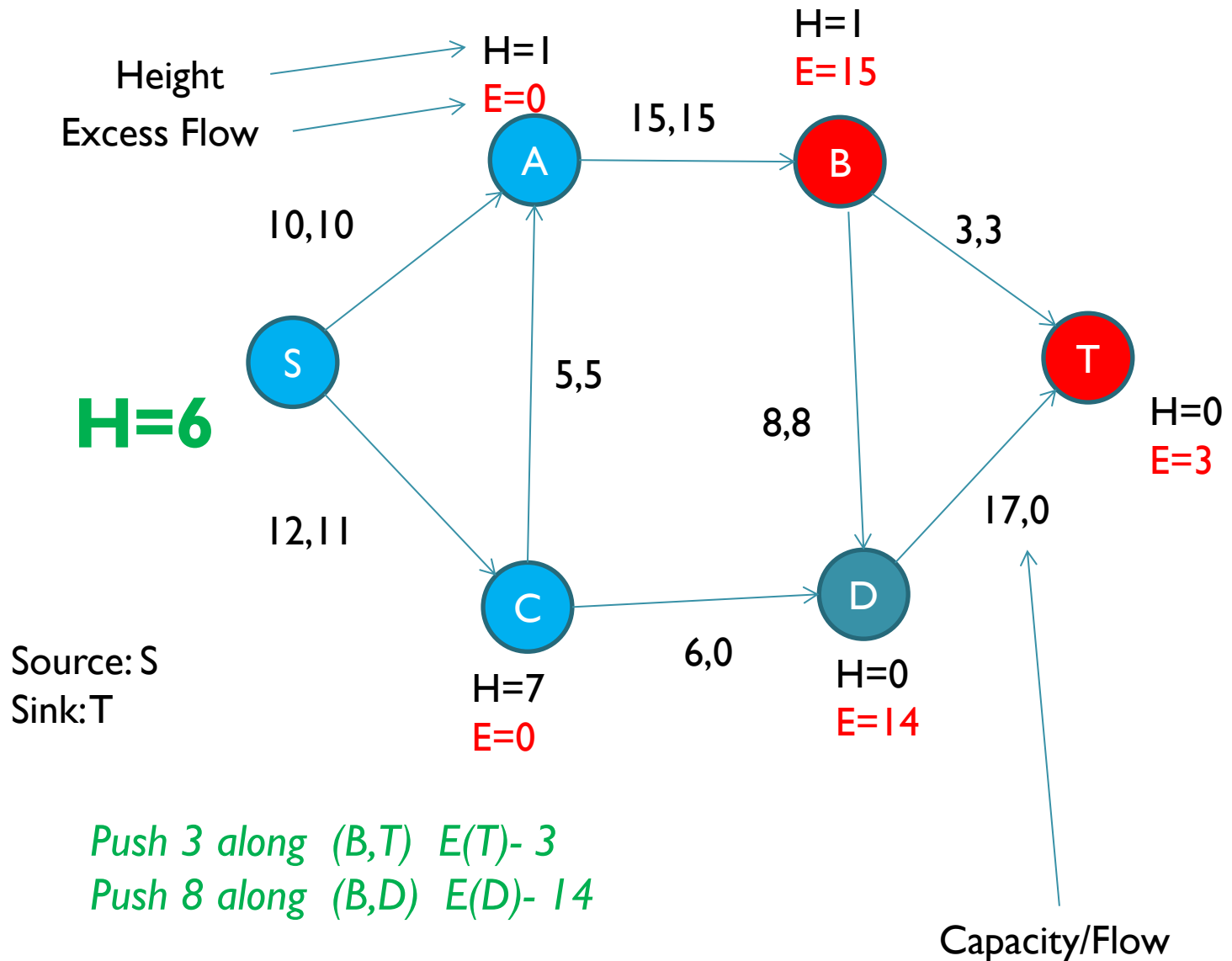
Push Relabel- RELABEL OPERATION C WITH $H=7$



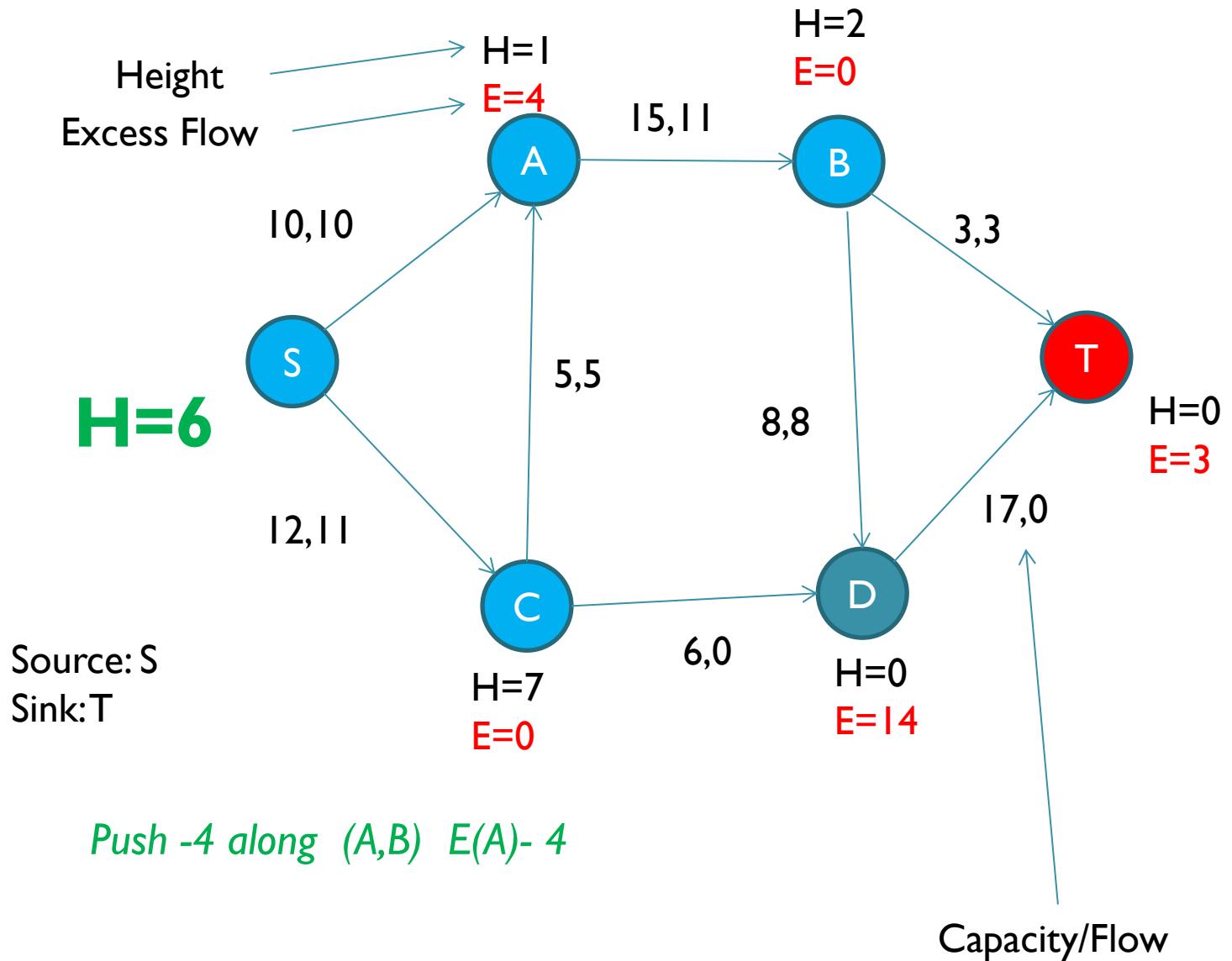
Push Relabel- RELABEL OPERATION A WITH $H=1$



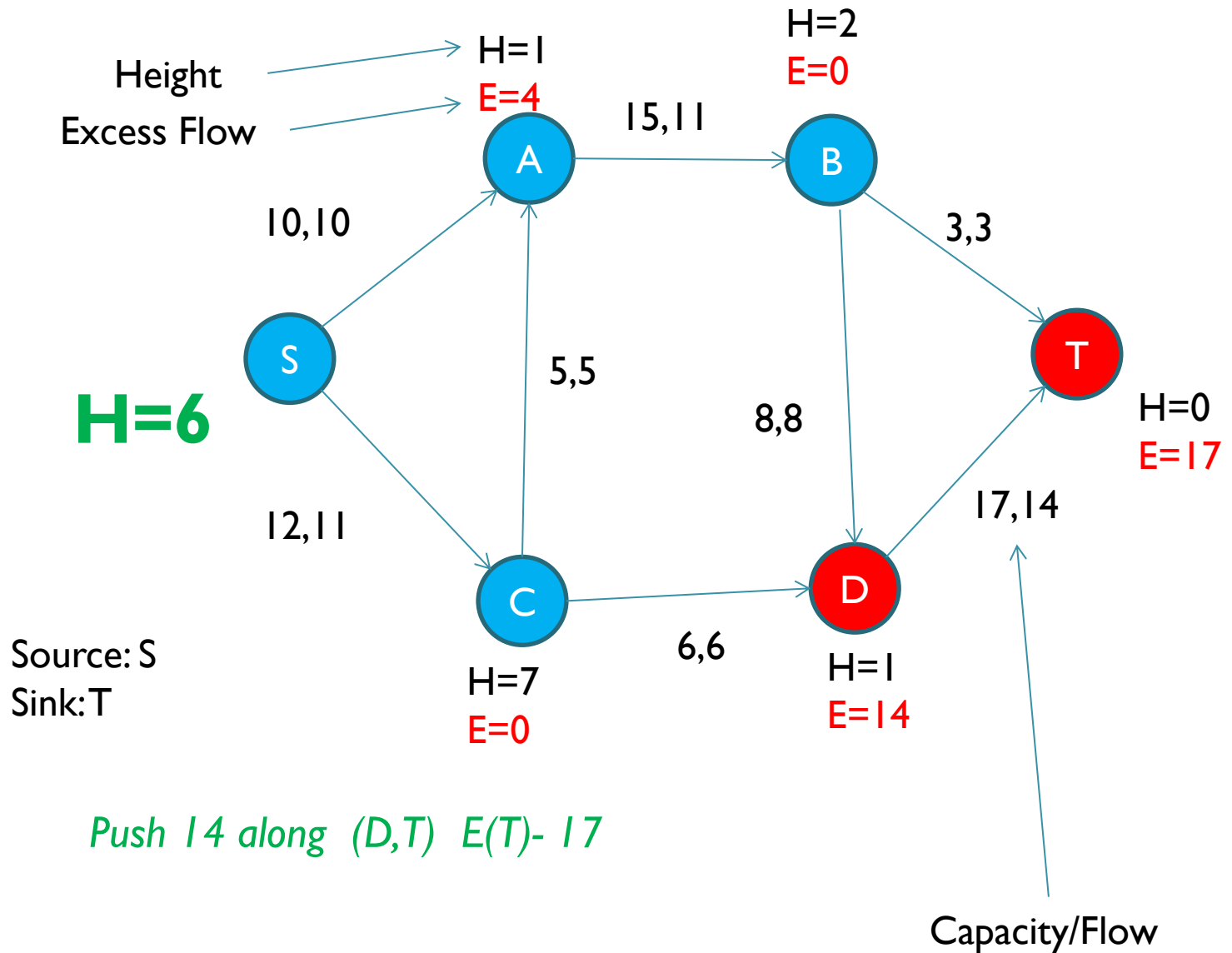
Push Relabel- RELABEL OPERATION B WITH $H=1$



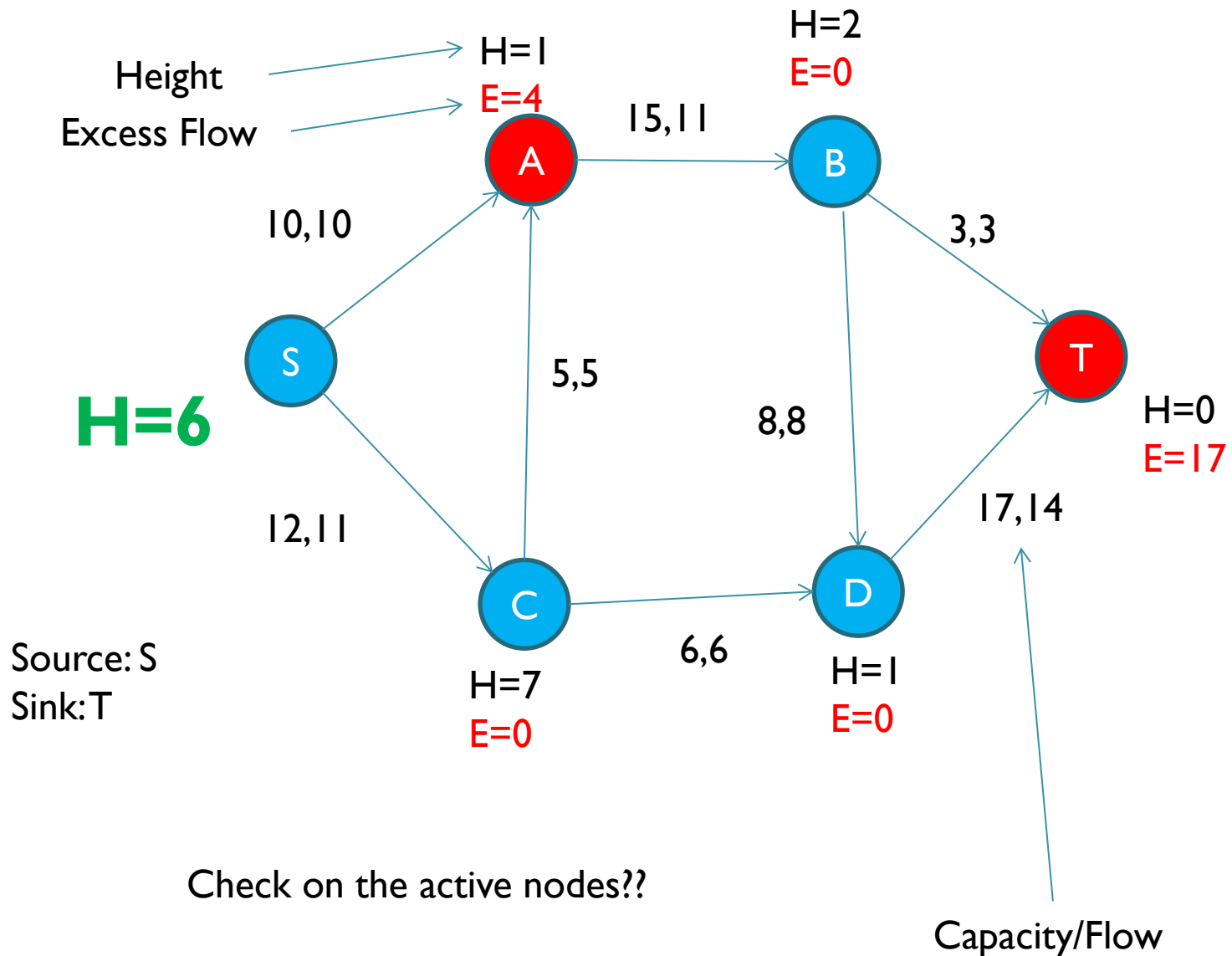
Push Relabel- RELABEL OPERATION B WITH $H=2$



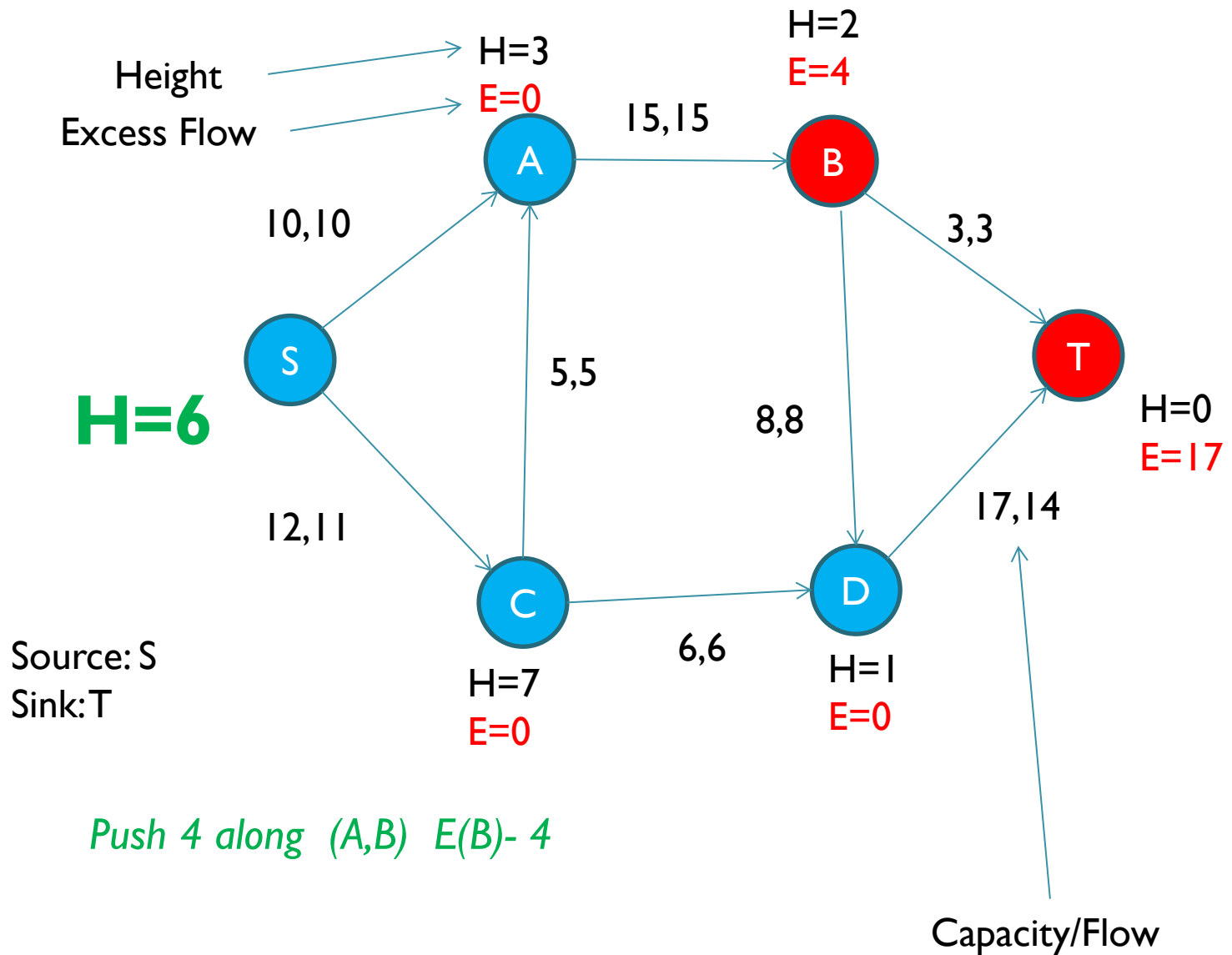
Push Relabel- RELABEL OPERATION D WITH $H=1$



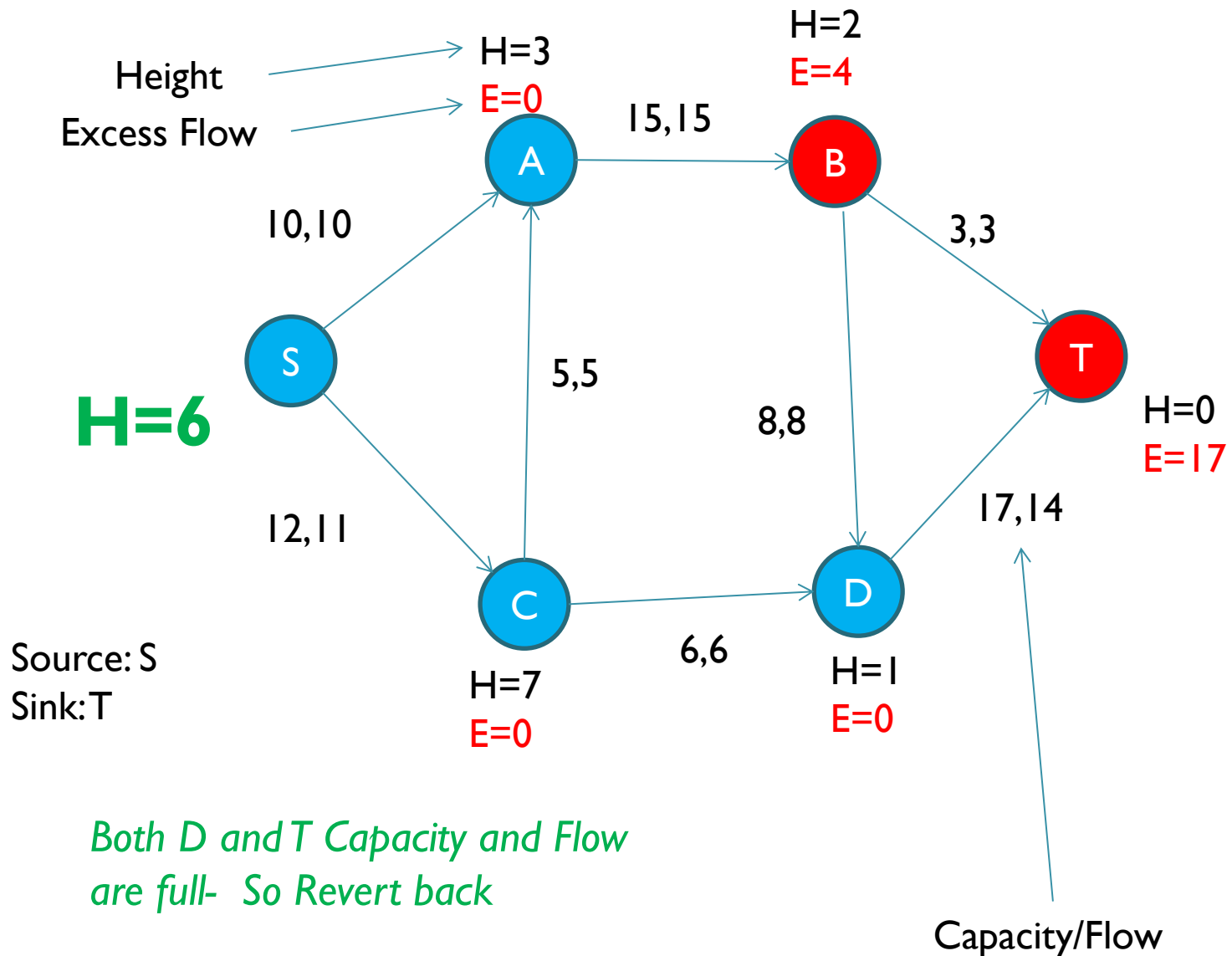
Push Relabel- Check on Active nodes- Here A



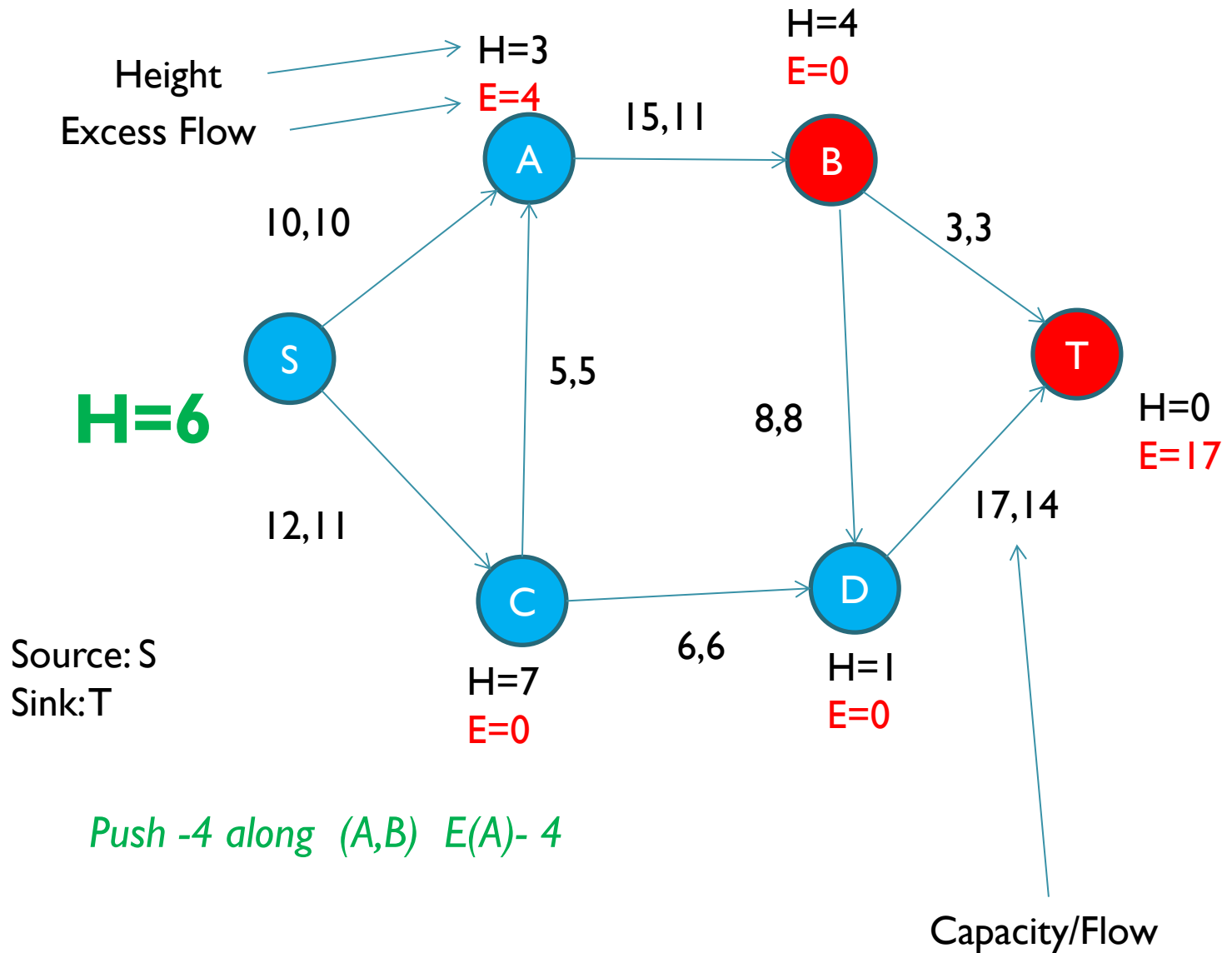
Push Relabel- RELABEL OPERATION A WITH 3



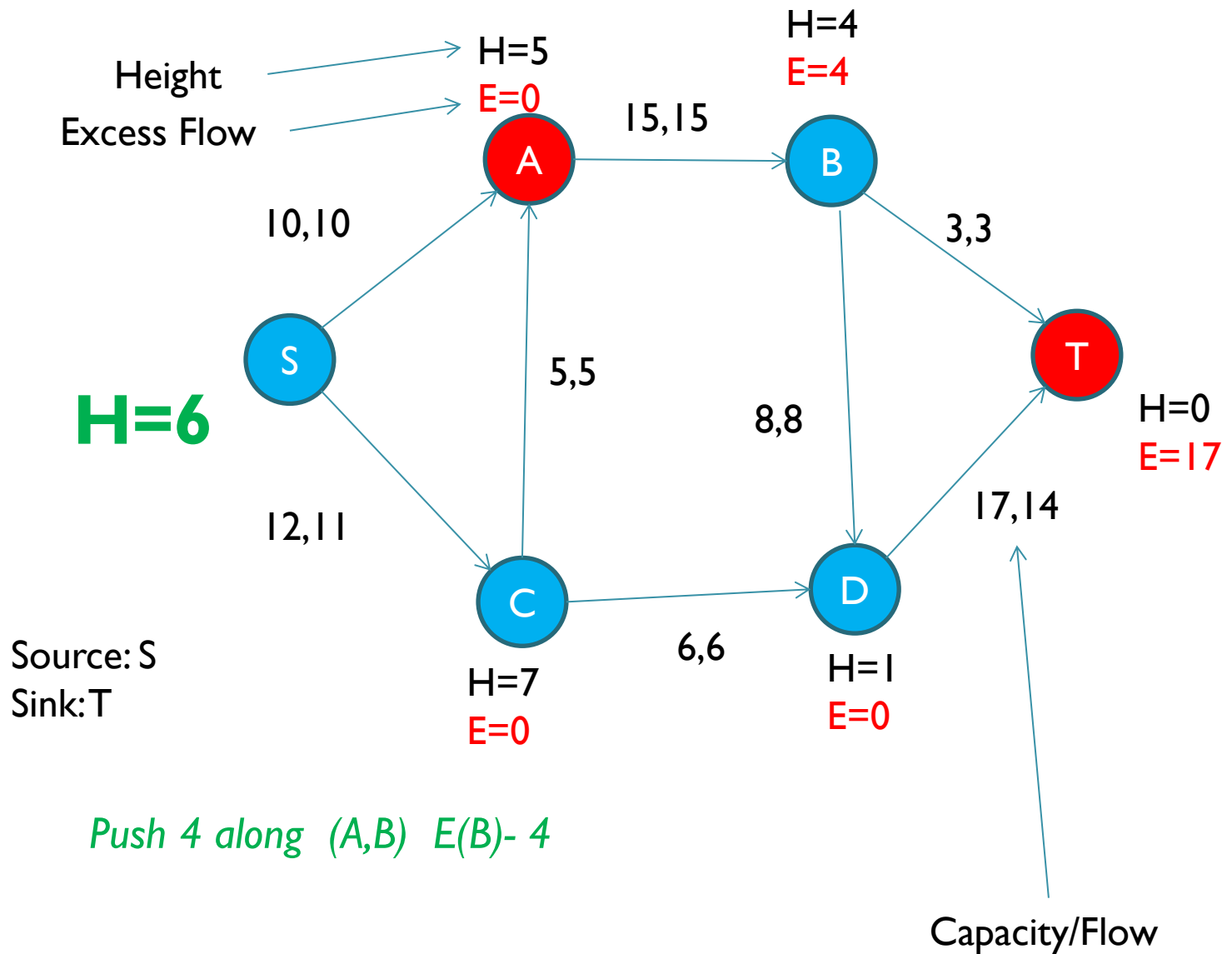
Push Relabel- What's next from B??



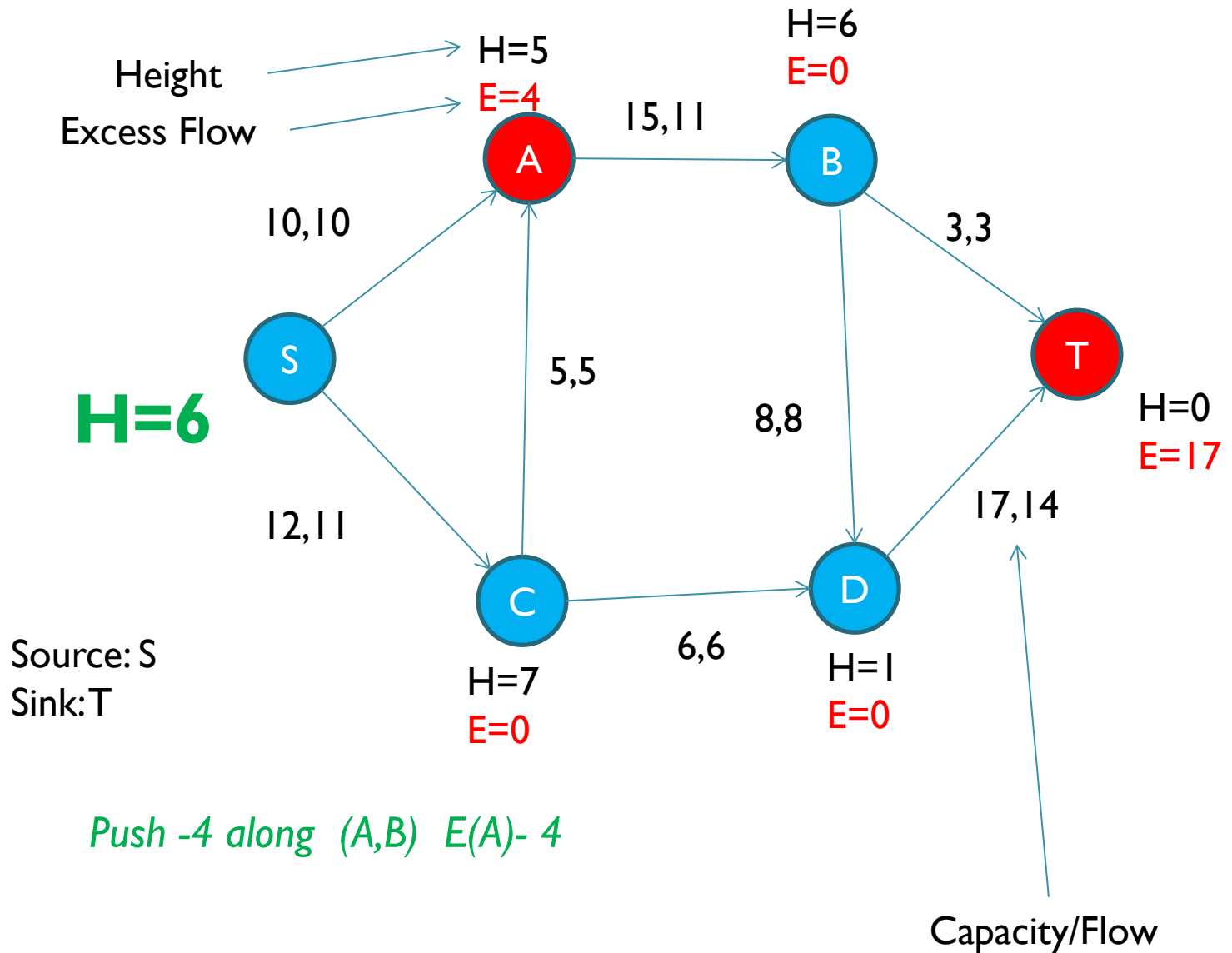
Push Relabel- RELABEL OPERATION B WITH 4



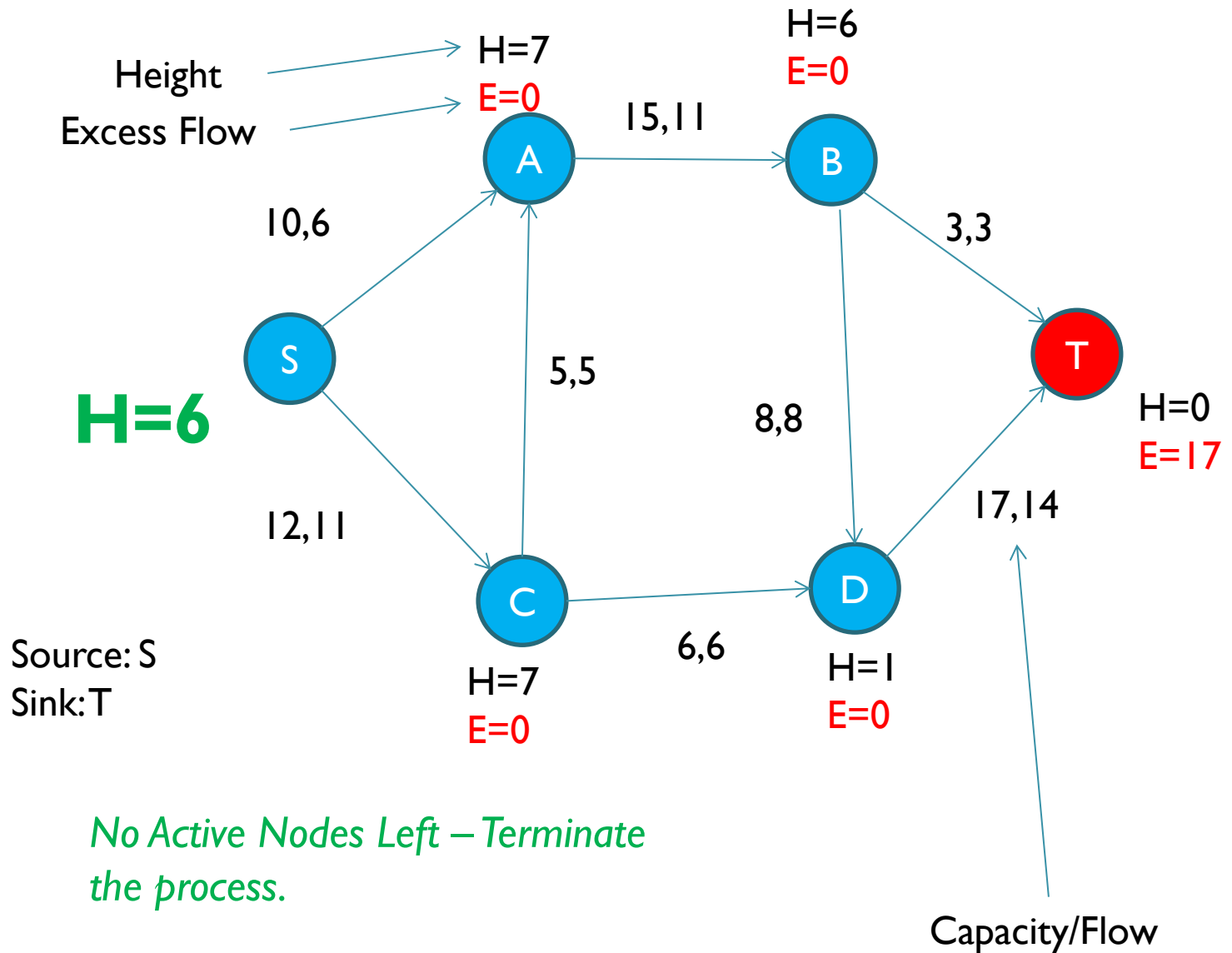
Push Relabel- RELABEL OPERATION A WITH 5



Push Relabel- RELABEL OPERATION B WITH 6



Push Relabel- RELABEL OPERATION A WITH 7





Thank you